Stress Signal

The Team

Riley Campbell	Caylum Blenkhorn	Zichen Zhang
● Original concept ● Programming, primarily the player and weapons system ● Asset creation ● UI	● Level design / Generation ● Game logic	● Enemy AI ● Enemy Animations ● Enemy Spawner

Development Plan

Timeline

Week 1 & 2	Week 3 & 4	Week 5 & 6	Week 7 & 8	Week 9 & 10	Final Submission
Project Setup	**Combat**	**Objectives**	**Refining Core Gameplay**	**Missions System and UI**	**Polish**

• Set up Git
• Basic player functionality
  ○ First person camera
  ○ Movement
• Level generation system
  ○ Create modules
  ○ Factor in objectives
  ○ Enemy spawn locations
  ○ Other variation considerations
	• Weapon system
	  ○ Shooting mechanic
	  ○ Weapon types
	  ○ Weapon modifiers
	• Enemy NPCs
	  ○ Basic AI (path, attack)
	  ○ Spawning
	• Player and enemy stats
	  ○ Health
	  ○ Damage
	  ○ Death
		• Mission generator started
		  ○ Random objective/s
		  ○ Difficulty
		  ○ Length
		• Ship systems
		  ○ Life support room
		  ○ Generator room
		  ○ Navigation room
		  ○ Fixing systems
		• Basic gameplay loop
		  ○ Completable missions
		  ○ Combat integrated
			• Multiple weapons types
			  ○ Pistol
			  ○ Shotgun
			  ○ Assault Rifle
			• Upgrades and equipment
			  ○ Loot drops
			  ○ Developed player stats
			  ○ Stat upgrades and special equipment
			• More enemy types
			  ○ Melee
			  ○ Flying
			  ○ Shooting
			  ○ Modifiers
				• UI for mission selection
				  ○ Random set of missions
				  ○ Difficulty, scale, objective
				• Rewards
				  ○ Gain reputation points based on mission type and performance
				  ○ Unlock content through reputation level
				• Other UI
				  ○ Menus
				  ○ HUD
					• Add content
					  ○ Enemy types/modifiers
					  ○ Weapons
					  ○ Weapon modifiers
					  ○ Equipment
					  ○ Level variation
					• Finalise sound and music
					• Finalise UI elements
					• Clean up
					  ○ Models
					  ○ Textures
					  ○ Levels

Planning

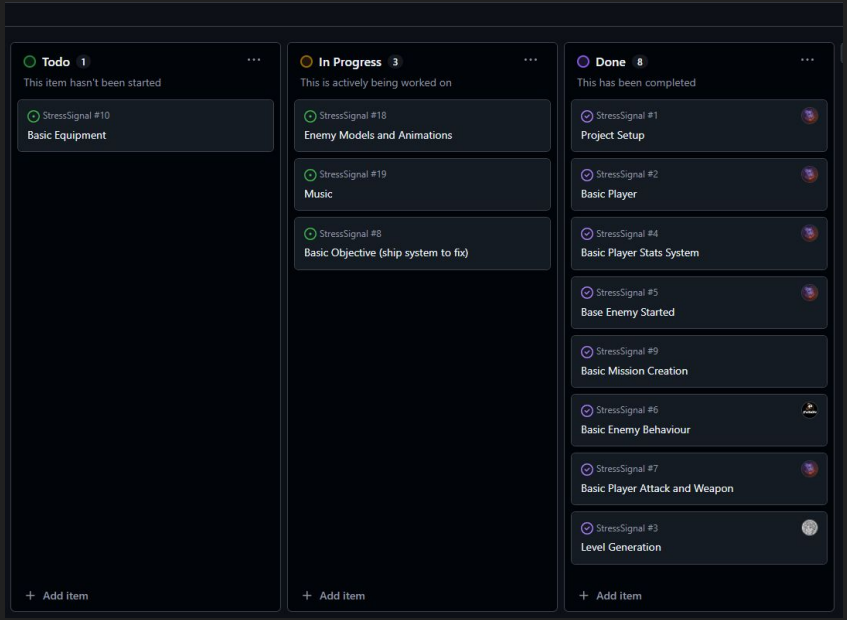
Set up a GitHub project for managing tasks.

Individual tasks were too big and should have been broken down.

We didn’t end up using it much.

Tasks were created based on priority each week.

Focused on direct communication through Discord.



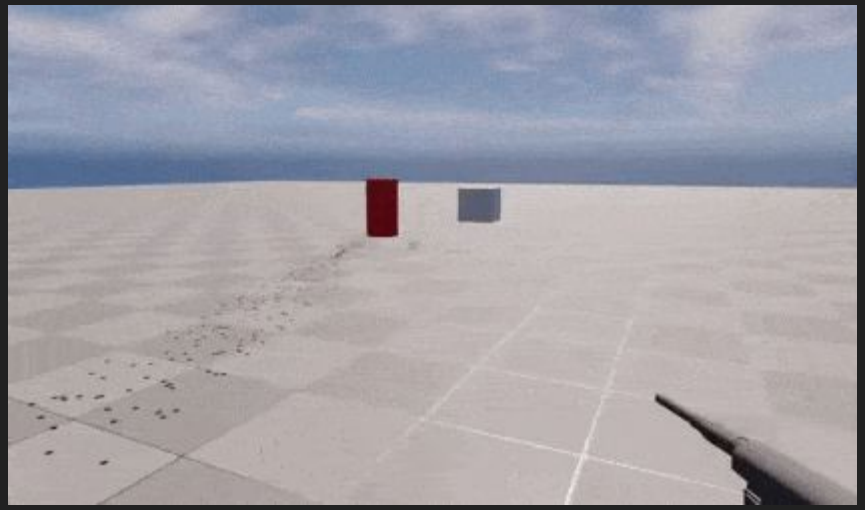
GitHub Project

Approach

The idea for Stress Signal was always to create a game that was easy to change and add to.

This was one of the ways the game would have a lot of potential replayability.

- Easy to add new weapons through a data table.
- Modular procedural level generation.
- Easy to modify enemies and create variants with different stats.
- Variety of mission types which can be added to.



1000 pellet shotgun

The Challenge

Randomness is hard to control. Things that have more possible variations return more unwanted results.

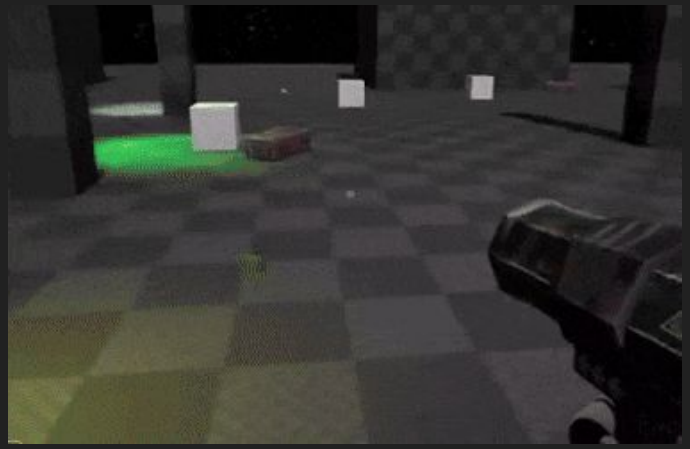
Creating systems that are easy to modify and add to can take up as much time as putting together all of the content from scratch.

Limits the structure of the game, it’s hard to make decisions about what the player will encounter.

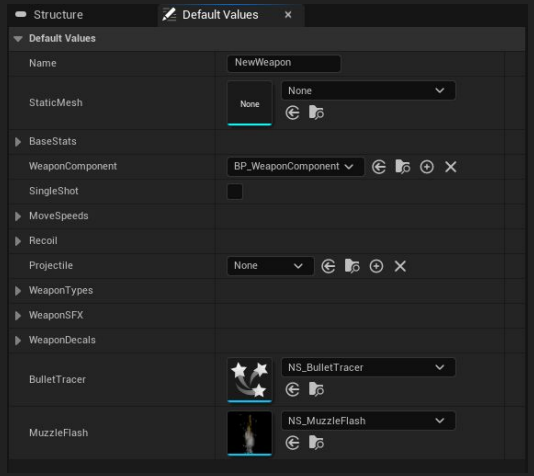
Weapons System

The weapons system works using a few important elements:

- Weapon Struct
- Weapon Data Table
- Weapon Component
- Weapon Actor



Rocket launcher jump into shotgun



Weapon struct

MISSION SELECTION
PLAYGROUND
MISSION SELECTION

STRESS SIGNAL

PLAY

SETTINGS

QUIT

THE SILENT OX

HARD

SHORT

EXTERMINATION

THE HAPPY FROG

MEDIUM

SHORT

EXTERMINATION

Click for Mouse Control

MISSION FAILED

LOBBY

MAIN MENU

Ship Generation Basics

The ship is build using a Minimum Spanning tree between randomly placed rooms within a predefined area.

- Ships always have a minimum of 5 rooms

- A room is a custom class that contains a multitude of different room 'types'

- 5+ Rooms are randomly placed within a given bounding box and assigned a 'type'

- once enough rooms have been placed, Delaunay Triangulation is run over all rooms to create a structure of edges for potential hallways to spawn

- Prim's Algorithm is then run to produce a minimum spanning tree to ensure all rooms are at least accessible from any given location.

- Extra hallway edges are then added to create loops across rooms for level variation

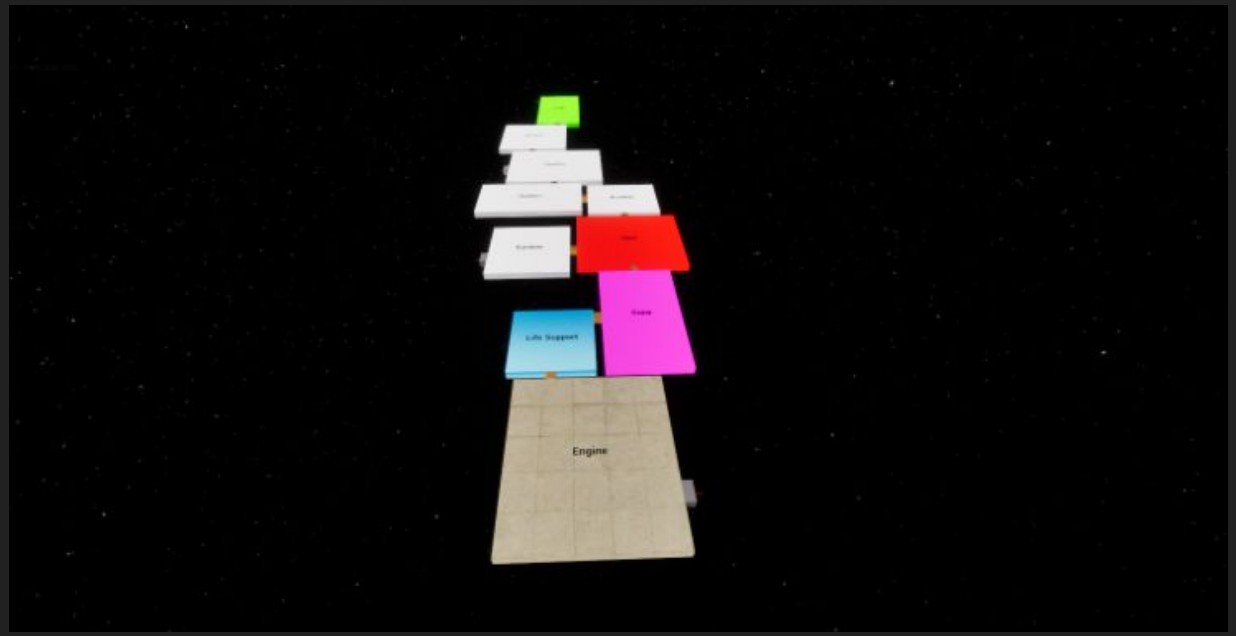
- Hallways are placed into the world among all edges chosen

- Once completed, rooms and halls are then 'built', which adds walls / doors to the room / halls.

Early Iterations

Initial Ship generation:
Engine room is first placed, then rooms can ‘grow’ off any randomly chosen room. (Depreciated early in favour of MST)

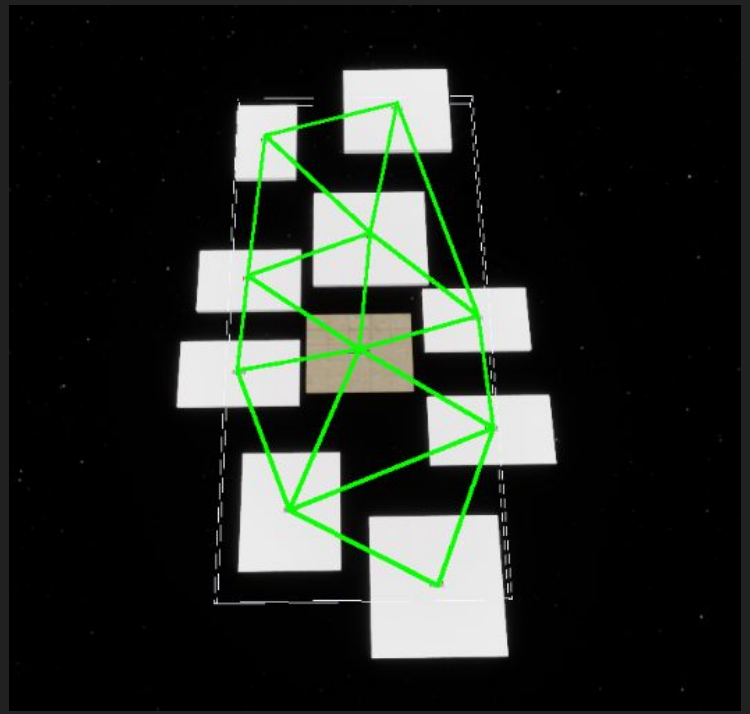
This method was initially useful as doors were always calculated, and every room was already accessible. This however doesn’t look great, isn’t within any bounds, and can lead to funky cactus like shapes

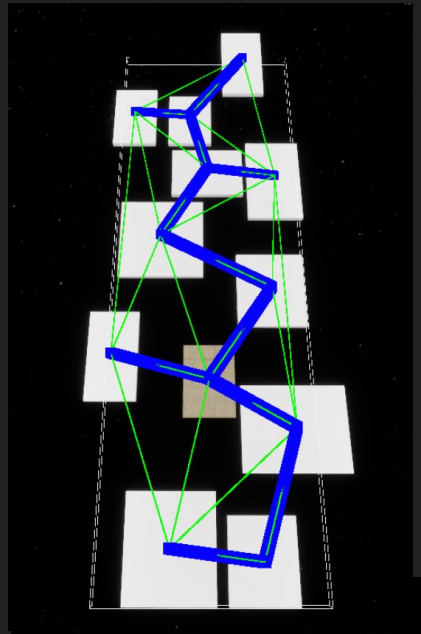


Similar ship generation with added stars in background

Connecting Rooms

Delaunay Triangulation. Rooms can only spawn inside a given bounding box, cannot overlap other rooms, and are tightly packed. If a room cannot be placed because it is either outside the box or overlapping another, it will not be spawned, and a ‘fail’ counter will increase, on reaching 200 fails, and not enough rooms are placed, rooms will be allowed to spawn slightly outside the box.

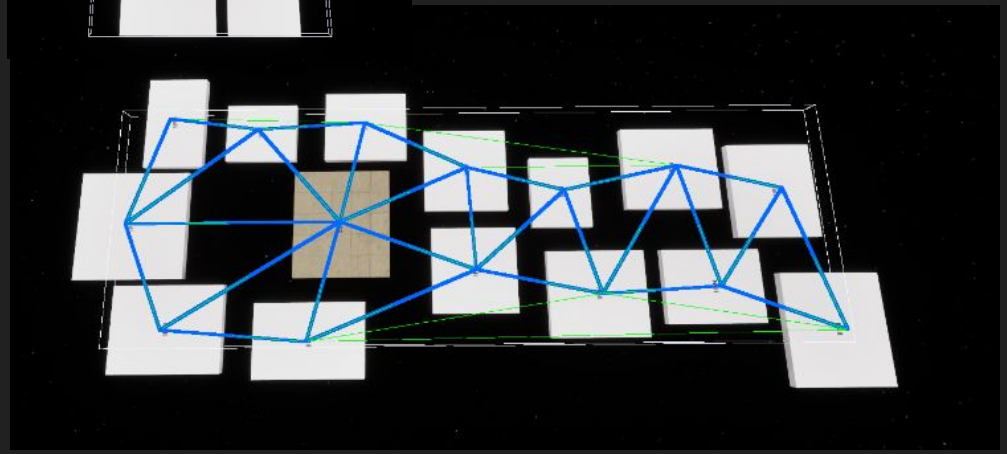




(Left) Prims Algorithm generating a Minimum spanning tree. The Engine (beige) is used as the root of the tree

(Below) Modified Prim's Algorithm. As an example, only ‘reasonable’ hallways are generated.

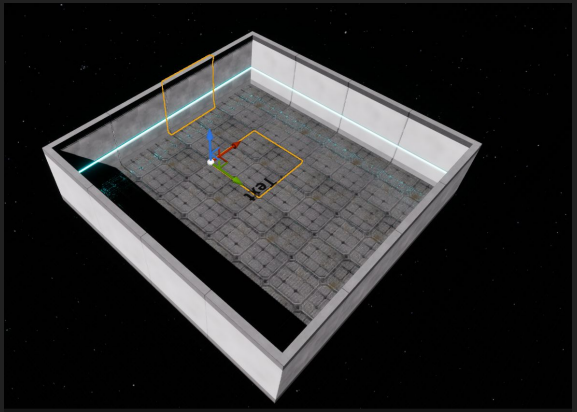
‘Reasonable’ being not intersecting another room, not too long, not outside ship bounds



Walls, Doors, and Stuff

From here on, the MST paths are used to generate hallways between rooms, walls and doors are placed accordingly

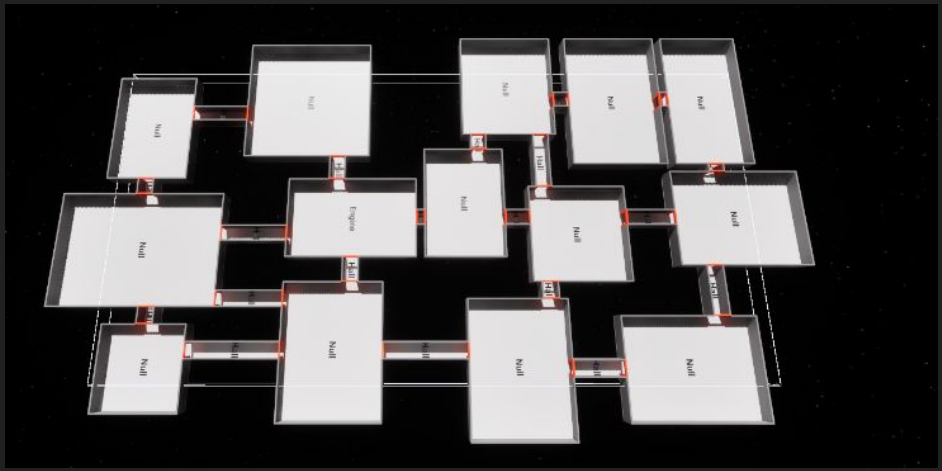
Door locations are generated first, then an instanced static mesh fills in each of the 4 (2 for Hallways) walls of a room, skipping a spot if a door is already existing in a location. As everything is grid based, locations are easy to calculate.



Dependant on Difficulty, mission type, and mission length, different amounts of enemies and nests are placed throughout the ship in rooms, with the spawn room always being empty.

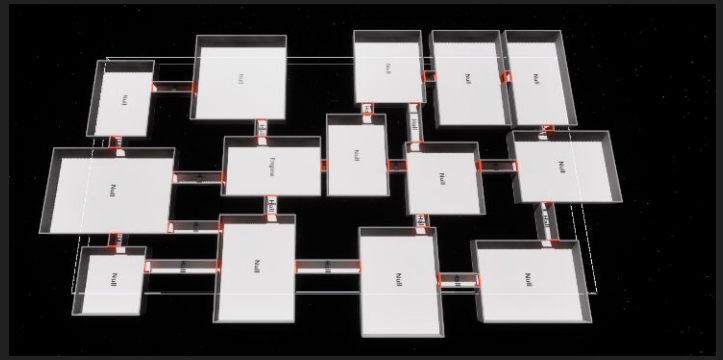
The grid has also been scaled up from 100x100 (bottom left and previous images), to a 400x400 grid

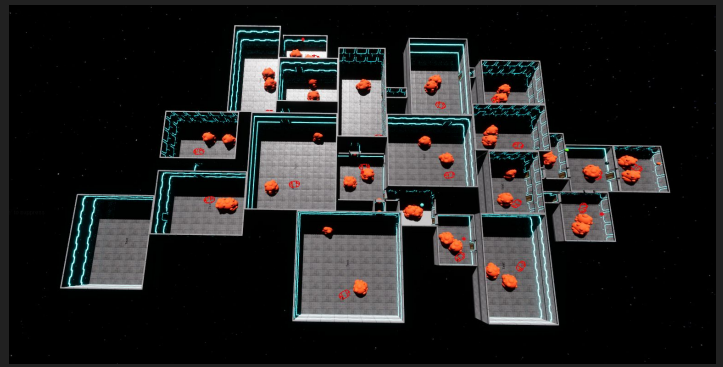
As seen below, this ship has all 5 required rooms … , and a cargo bay with twice the height.


Null
Null
Null

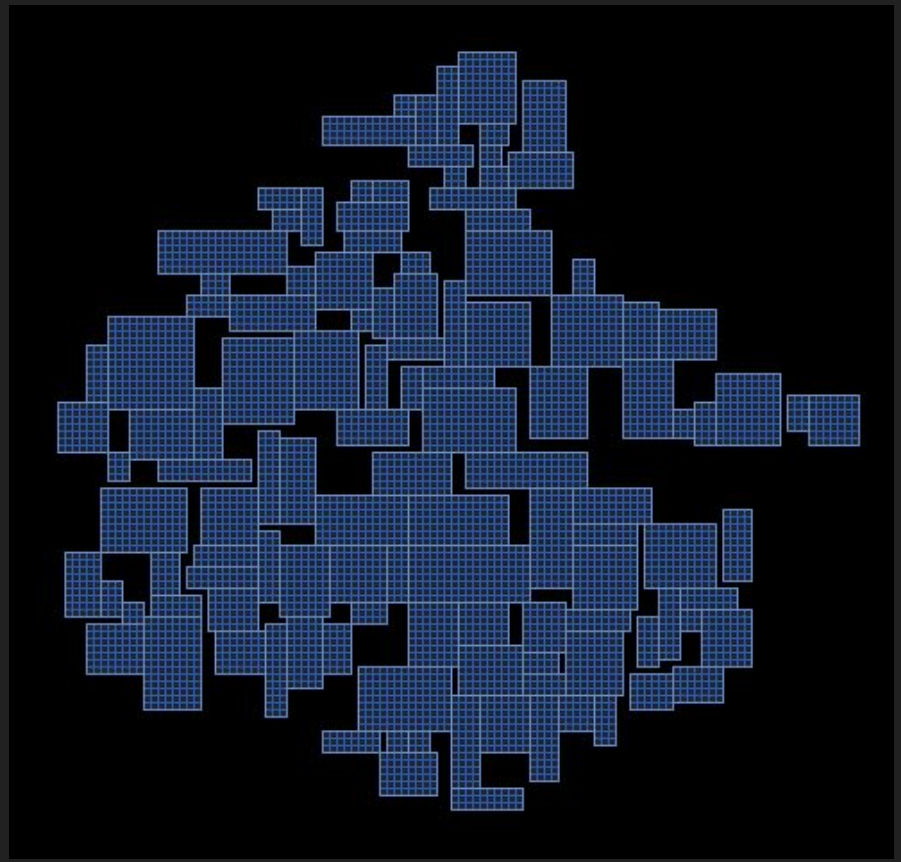
Rooms are too spaced out and can often lead to situations where a multidirectional hallways is needed to connect two rooms.

This generation is more appropriate for a dungeon or space station perhaps; to remedy this, each room is no longer spawned randomly within the bounds of the ship, and instead each time a room is spawned, it is randomly adjusted until it no longer overlaps with another room, leading to far more compact and cleaner ships. (and no while loops!*)

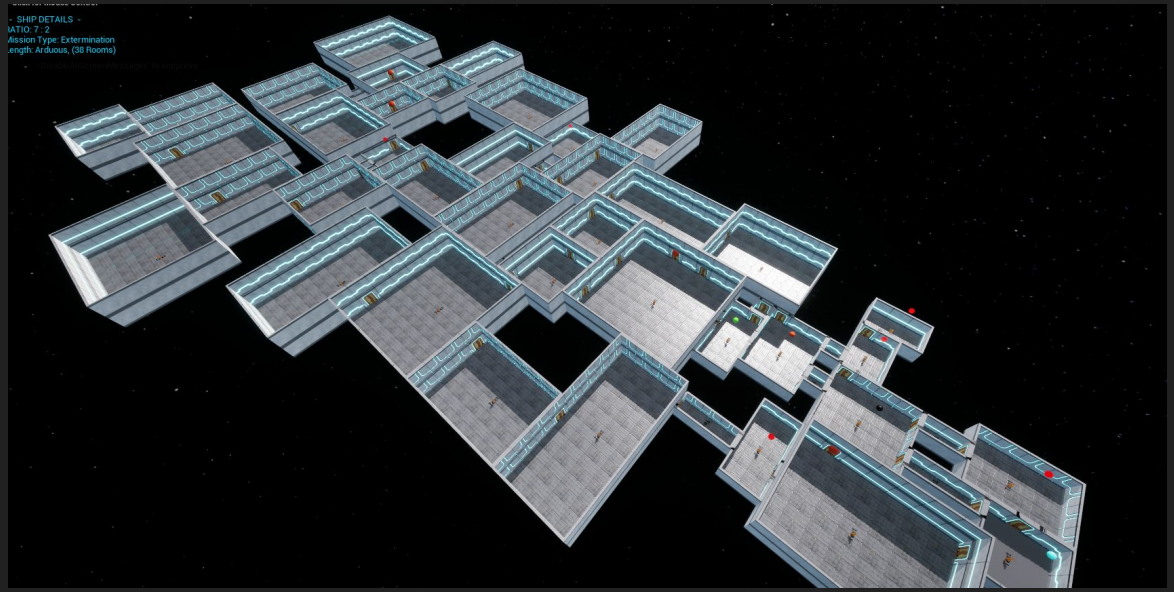




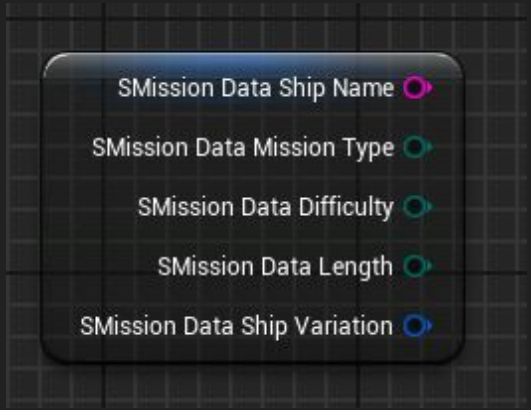
*



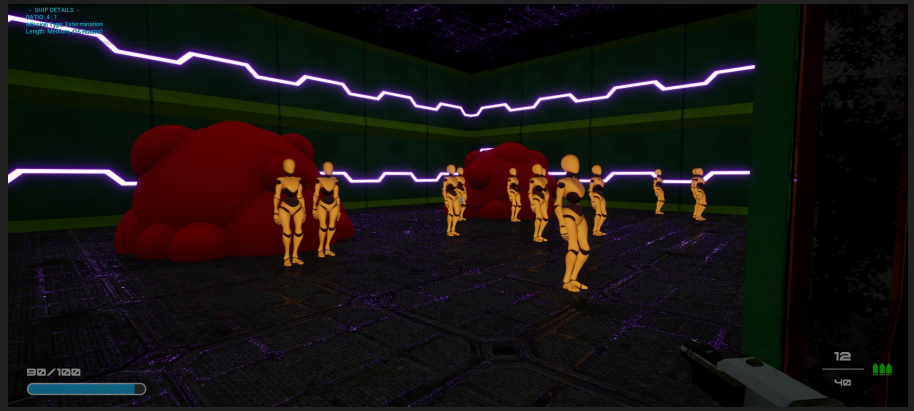
Animation from:
https://www.gamedeveloper.com/programming/procedural-dungeon-generation-algorithm



By taking in gamemode specified parameters, the ship and it's room are generated accordingly. Type, Difficulty, and Length Enums all influence the maximum rooms, enemies, and nests found throughout the ship



The Variation Struct controls things like wall colour and lighting



Development

–Navigation: NavMesh, AI Move To

–Attack (melee and ranged attack): Attack distance, Accept Radius

–Animations: animation montage, state machine

–Basic Enemy Spawner: structure, data table, construction script

Reflection

–Smarter AI system

–Animation layering

–Compatible spawner

Result